

MSc Data Science Project

7PAM2002-0901-2024

Department of Physics, Astronomy and Mathematics

Data Science FINAL PROJECT REPORT

Project Title:

Housing price prediction using machine Learning and
Deep Learning

Student Name and SRN:

Kushal Kumar

Supervisor: Dr Sophie Koudmani

Date Submitted: 6 January 2025

Word Count: 6860

GitHub address- https://github.com/kushal-jaat/ASD-assignment-3/blob/main/housing_price_prediction.ipynb

Second Link - <https://github.com/kushal-jaat/ASD-assignment-3/blob/main/salesdata.ipynb>

DECLARATION STATEMENT

This report is submitted in partial fulfilment of the requirement for the degree of Master of Science in Data Science at the University of Hertfordshire.

I have read the guidance to students on academic integrity, misconduct and plagiarism information at [Assessment Offences and Academic Misconduct](#) and understand the University process of dealing with suspected cases of academic misconduct and the possible penalties, which could include failing the project module or course.

1. I certify that the work submitted is my own and that any material derived or quoted from published or unpublished work of other persons has been duly acknowledged. (Ref. UPR AS/C/6.1, section 7 and UPR AS/C/5, section 3.6).

I have not used chatGPT, or any other generative AI tool, to write the report or code (other than where declared or referenced).

I did not use human participants or undertake a survey in my MSc Project.

I hereby give permission for the report to be made available on module websites provided the source is acknowledged.

22021423

Kushal Kumar

Kushal Kumar

UNIVERSITY OF HERTFORDSHIRE

SCHOOL OF PHYSICS, ENGINEERING AND COMPUTER SCIENCE

Abstract

Understanding and predicting the price of properties is essential and still, it remains a challenge due to the diverse market and factors influencing it. In this project I explored how machine learning and deep learning models can be helpful in predicting the prices of housing and rental properties, addressing the challenges of capturing the complex relation of real-world data. I used two datasets first one real estate sales dataset from the State of Connecticut and a nationwide USA housing rental dataset. To address the challenges I handle missing values, outliers and skewed distribution the data was pre-processed properly with the techniques of log transformation, statistical outliers removal and encoding categorical variables. And scaling the numerical feature. After this step, it was time to make the models I made a baseline linear regression model implemented then some advanced models like LightGBM and Random Forest as well as deep learning models (ANNs), were used to capture the complexity of non-linear patterns. In addition to offering a solution in this housing sector of industry, this study shows how everything was handled carefully with advanced machine learning and deep learning techniques, which can open the door for future optimization and practical implementation while providing insightful trends in the prices of the housing and rental market.

Contents

1. Introduction	07
1.1 Aim.....	08
1.2 Current Industry Practices.....	08
1.3 Significance of the Project.....	08
1.4 Research Question.....	08
2. Background	
(Literure).....	10
3. Dataset	
3.1 Origin and Collection of Data.....	12
3.2 Dataset selection.....	12
3.3 Ethical Issues.....	12
4. Methodology	
4.1 Introduction.....	13
4.2 Data Features for USA Housing Dataset.....	13
4.3 Data Feature for Connecticut Real Estate Sales Data.....	14
4.4 Dataset Overview for the sales data.....	15
4.5 PRE-PROCESSING.....	17
4.6 Data Split.....	22
4.7 LinearRegression.....	23
4.8 Random Forest Model.....	23
4.9 LightGBM Model.....	23
4.10 Artificial Neural Network (ANN).....	24
4.11 Challenges.....	26
5. Results	
5.1 Model prediction	27
5.2 USA Housing Listing Data Prediction.....	27
5.3 Connecticut Real Estate Sales Data Prediction.....	29
6. Analysis and Discus-	
sion.....	32
7. Conclusion.....	32

Introduction

Predicting housing prices accurately is a very important task estate industry. It impacts directly property valuation, investment strategies, urban planning, and the housing market or depends on numerous factors such as property attributes, location, economic trends and market demands, which deal in highly complex and non-linear ways by Selim, (2009). Traditional methods like hedonic regression and linear model, while foundationally are not sufficient to capture the complexity present in the modern dataset. The advancement of machine learning and deep learning techniques has given Innovative tools for solving these challenges, enabling the analysis of large and diverse datasets with proper precision Adetunji et al., (2022).

This project will explore the potential of machine Learning and deep learning techniques for predicting the housing prices in both datasets. The sales dataset includes key attributes such as property type, prices, town, sale ratio, residential type, etc., whereas the rental data includes some of the best key attributes like type, sqfeet, beds, bath, cats allowed, etc. By using these datasets, the project aims to uncover patterns and complex relationships that traditional models often overlook, delivering more accurate and interpretable predictions.

Preprocessing is the most important part of this or any other modelling project, ensuring data quality and consistency for modelling. Techniques such as log transformation, feature scaling and one-encoding were used to handle skewed distribution, normalize values and prepare categorical variables for modelling. These preprocessing steps align with the best practices as evidenced by Zhan et al. (2023), emphasising their role in optimizing ML and DL model performances. Additionally, in this, I also handled common challenges such as missing values and outliers ensuring that models are trained on reliable and representative data.

Four types of models were selected for this project based on their proven effectiveness in predictive analytics. As a baseline model, I used linear regression. The second one was random forest and light GBM and the last one was Artificial Neural Networks (ANNs). Random Forest. Is used as a benchmark model. Due to its robustness and ability to identify the importance of individual features. As demonstrated by Adetunji et al. (2022). Light GBM. Known for its computational power and performance with structured data. and is used to enhance the prediction speed and accuracy as highlighted by Lahmiri et al. (2023). ANNs with their capacity to model non-linear interaction and complex relationships. provide the deep learning perspective required for understanding intricate dependencies within the data Selim, (2009).

This study is not just limited to testing models it also evaluates their performances using metrics such as root-squared mean error RMSE, Mean Absolute Error MAE AND R-squared (R^2). These metrics allow for a comprehensive assessment of each model's

strengths and limitations, guiding the most effective approach for housing price prediction.

1.1 Aim

Investigate how the cutting-edge ML and DL models, including ANNs, LightGBM, and random forest, to be used in predicting the value of housing and sales price. Provide pre-processing techniques that handle the issues such as outliers and missing values. assessing the models on various datasets to identify the best strategy for practical uses in real-world applications.

1.2 Current Industry Practices

The real estate industry has used a lot of predictive analytics to forecast housing prices and rental values. Traditional models like linear regression and hedonic pricing remain popular but struggle with the complex dataset in modern times. In recent years, machine learning techniques like random forest and gradient boosting machines have gained a lot of names in the market because of their robustness and ability to handle large, structured datasets. Deep learning models, including artificial neural networks (ANNs), are coming into the light as powerful tools for capturing non-linear relationships and spatial-temporal dynamics. However, challenges such as handling missing data, ensuring interpretability, and scaling these models into the market have limited their widespread adoption Zhan et al., (2023). This project addresses these limitations by improving preprocessing techniques and advanced predictive models to develop a scalable and interpretable framework.

1.3 Significance of the project

This study bridges the gap between traditional statistical methods and advanced AI-driven approaches and tries to solve complexities in real estate datasets. By doing these methods of machine learning and deep learning techniques the project not only improves prediction accuracy but also provides insights into the factors driving the housing prices and rental value. These insights will help the stakeholders including buyers and sellers provide the best decision-making forecasting in the housing market.

1.4 Research question

I.

I tried to investigate how machine learning and deep learning can be optimised to improve the accuracy of housing price prediction.

To improve prediction accuracy. I used models such as random forest, LightGBM, and ANNs, checking their capabilities to handle non-linear relationships and complex patterns. Baseline performance was established with linear regression and advanced models were optimised through hyper tuning. Getting good results evaluated using RMSE, MAE AND R2 metrics.

II.

preprocessing techniques are most effective in addressing common data challenges in housing datasets?

I used imputation for missing data, IQR for outliers, log transformations for normalising skewed distributions, and one-hot encoding for categorical variables as part of the preparation step. To improve model stability, Standard and Min-Max scaling methods were used to scale numerical features. These collaborative initiatives demonstrated the potential of ML and DL in tackling practical issues in housing markets by ensuring accurate predictions, interpretable insights, and dependable data.

This research question is focused on understanding how these models will improve in the way of predicting prices. The goal isn't just about building better models; it's about solving real challenges that people face in the housing market every day. Whether it's helping investors make smarter decisions, property managers plan more effectively, or policymakers create better housing policies, I hope this work will contribute something meaningful. By finding the answers to these questions.

BACKGROUND

2 Literature

The prediction of housing prices has been a key area of research due to its importance in real estate, urban planning, and investment decision-making. However, there has been complexity in the non-linearity of factors such as location, property feature, and market flow which creates a big challenge for the traditional models but the recent advancements in machine learning and deep learning have provided a solution to address these challenges, giving to analyse the large and complex datasets with better accuracy and interpretability

For this project, I conducted a review of research papers more than 6 and selected only five peer-reviewed studies that are directly related to my work. These papers were chosen based on their machine learning and deep learning techniques, structured and spatial data use, and importance in preprocessing and feature engineering. Each of these papers gives a different intel into different aspects of housing prediction, including model development, data preprocessing and evaluation metrics. The following section will highlight the contributions, relevance and limitations in their field.

The study done by Adetunji et al. (2022) explored the use of random forests for predicting housing prices, underlining its ability to handle structured datasets and rank feature importance. The study utilized a comprehensive property transaction dataset with attributes like property size, location, number of rooms, and type. Preprocessing steps include handling missing values, normalizing features and encoding categorical variables, and ensuring data quality for training the model. Random forest was used to check the influential predictors and predict housing prices, with the result showing high predictive accuracy and robustness, achieving an error margin of around 5% positive and negative. The authors also included that RF is effective for structured data and highlighted its interpretability, particularly through feature importance ranking. However, this study's limitation includes a lack of consideration for spatial data, which are critical in capturing real-world complexities.

The study shown by Selim (2009) compared hedonic regression with artificial neural networks to examine their effectiveness in predicting housing prices, particularly in modeling non-linear relationships. The dataset included property attributes like size, location and amenities, which were from the 2004 household budget survey in Turkey. The method he used to develop two predictive models first one a hedonic regression model as the traditional baseline and second one an ANN model with multiple hidden layers for capturing complex patterns. The results showed that ANNs have performed better than hedonic regression both in accuracy and prediction. The study concluded that neural networks are the best choice for real estate data, although it lacked detailed hyperparameter tuning, which could further make the ANN model perform better. This paper is highly relevant to my project as it used the ANNs to capture intricate dependencies in housing data.

The study by Zhan et al. (2023) introduced a hybrid framework that combines Random Forest and deep neural networks to predict housing prices. In this study, they used the dataset's geographic coordinates, property attributes, and transaction dates. Giving it to understand the spatial and temporal dynamics in the housing market. They used the random forest to handle this structure data. And DNNS to capture non-linear dependencies and time-based trends. Achieving, good performance. Compare to standalone models. The authors also suggested that multiple modelling approaches give better prediction accuracy and robustness. Although challenging, such as computational cost and overfitting were also seen. This study also aligns closely with my project objective of leveraging both ML and DL techniques to address the multifaced nature of housing price prediction inspired by this approach. I'm thinking to implement implementing a hybrid framework. Addressing these challenges. Through efficient data pre-processing and model optimization.

The study by Lahmiri et al. (2023) evaluated the performance of boosting assembly regression trees, support vector regression, and Gaussian process regression for housing prices. The study utilised a comprehensive housing data set and employed Bayesian Optimization, with a tenfold cross-validation framework to identify optimal parameter values. The results showed that boosting assembly regression trees performed better than other models by Getting the best accuracy. The paper concluded that Bayesian optimization is an important step to make models perform better. However, it is also seen that using these methods in complex datasets is challenging. This research directly informs the hyperparameter optimisation strategies in this project so that random forest, lightGBM, and ANN models can achieve better optimal performance.

The study conducted by Sharma et al. (2024) is a detailed comparison of machine learning models, including XGBoost, Random Forest, Support Vector Regressor (SVR), Multi-layer Perceptron (MLP), and Multiple Linear Regression, for housing price prediction. Using the Ames City housing dataset, the study highlighted the significance of property attributes and neighbourhood features in influencing housing costs. The results showed that XGBoost gives better results than all of the other models due to its ability to handle non-linear relationships and large datasets properly. The study also showed the importance of feature selection and preprocessing in achieving high accuracy. However, the authors also noticed that hyperparameters for the XGbosst are computationally expensive, which aligns with the challenges addressed in this project through efficient optimisation techniques like early stopping. This study supports the use of advanced ensemble methods like LightGBM in this project, validating their ability to achieve robust and accurate predictions for housing price analysis.

Datasets

3. Origin and Collection of Data

The **Connecticut Real Estate Sales Data** is a publicly available dataset sourced from [Data.gov](https://data.gov) and maintained by the Office of Policy and Management (OPM). It contains all real estate transactions in Connecticut with a sale price of \$2000 or above. The data set ranges from 2001 to 2022 and covers our variety. All property types, including residential, commercial, industrial. The data is collected and reported annually by the Grand list year, from October 1st to September 30th. The main goal is transparency and support for urban planning initiatives. It is also a public and non-federal licence. Permit to use it for academic research.

The **USA Housing Listings Data** taken from the [Kaggle](https://www.kaggle.com) website was provided by Austin Reese, this dataset aggregates rental listings from Craigslist which is the world's largest collection of privately sold housing options. It gets updated quarterly; the last update was in December 2024. The data set provides a broad view of rental properties capturing attributes like rent, price, property size, square ft number of bedrooms, bathrooms, parking options and other amenities. The data is taken to perform the task of analysis on housing trends across the United States. It is accessible to the public because of its public domain status and is ensured by the CCO license, which permits the use of it for educational purposes. Which comes under the ethics rule and standard.

3.1 Dataset selection

For this project. I selected two data sets, the Connecticut real estate sales data and the USA housing listing. I chose these datasets because they align perfectly with my objective of improving housing price prediction and addressing preprocessing challenges in real estate data. The Connecticut dataset is given over two decades (2001–2022) and contains more than one million records, including details like sale amount, assessed value, property type, and location. These are important which help to determine different factors which influence housing prices. The USA Housing Listings dataset contains around 385,000 rental property records, covering features such as rent price, property size, and amenities. Both datasets allow for border analysis. Which is important for creating robust machine learning and deep learning models.

3.2 Ethical Issues

This used two dataset which are publicly available on data.gov a government official website and Kaggle with aa public domain the datasets anonymized with GDPR compliance, and it is also permitted to use for academic under a creator common license. And because of there were not required UH ethical approval.

Methodology

4. Introduction

In this section, I have explained the tasks for the Connecticut Real Estate Sales Data and the USA housing listing. I have done a lot of preprocessing steps and divided them into two parts. First, I explained the sales dataset and their preprocessing, followed by the housing listing. After this, I explained my models. The first one is Linear regression as a baseline, the second one is the random forest and LightGBM, and the last is a type of deep learning model (ANN). I used RMSE, MAR and R^2 scores to check the model's performance and accuracy.

4.1 Data Features for USA Housing Dataset

The USA Housing dataset contains 384,977 entries and 22 attributes, showing detailed information about rental listings across the United States. In this there are three types of attributes numerical, categorical, and geographical each of these contributing to the analysis of rental market trends.

➤ List of Columns,

- Id is the number assigned for each listing.
- Url is the website link.
- Region is the area of the property.
- region URL
- Price is the price of the property.
- Type is the type of property e.g. apartment, condo, house.
- Sqfeet is the size of the property in square feet.
- Beds show how many bedrooms are available on the property.
- Baths are the number of bathrooms available on the property.
- Cats allowed shows that if cats are allowed or not 0 = No, 1 = Yes.
- Dogs allowed shows if they are allowed 0 = No, 1 = Yes.
- Smoking allowed whether you can smoke on the property or not 0 = No, 1 = Yes.
- Wheelchair access provides that if the property can be accessed by a wheelchair or not 0 = No, 1 = Yes.
- Electric vehicle charge tells if there EV charging facilities are available 0 = No, 1 = Yes.
- Comes furnished tells whether the property comes furnished or not 0 = No, 1 = Yes.
- Laundry options: does it come with these facilities or not w/d in the property
- Parking options: What type of parking e.g. off-street parking.
- Image URL is Linked to the image of the property.
- Description is the information about the property.
- Lat is the Latitude location.

- Longitude location.
- State to which state it belong.

4.2 Data feature for Connecticut Real Estate Sales Data

The **Sales Data** has 14 columns. These include unique identifiers, financial details, geospatial data, and categorical information about property characteristics. (Fig 1)

- Serial Number gives all the information assigned to each of the values.
- List Year in which year the property was listed I also dropped it
- Date recorded the year in which the transaction was recorded.
- Town the area in which the property is located.
- Address I dropped it.
- Assessed Value the property value determined before selling.
- Sale amount: The actual price at which the property can be sold. I also used it as a target variable.
- Sale Ratio it's the ratio of assessed value to sale amount, showing assessment accuracy.
- Property Type in this includes type of property residential, commercial, industrial, etc.
- Residential Type contains single family two family etc.
- Non use Code tells conditions affecting the property's Usability.
- Assessor remarks note from the property assessor I didn't use this because a lot of missing values.
- OPM Remarks notes from office of policy and management I also dropped it in the coding because of a lot of missing value
- Location geospatial information about the Property I also removed it because its missing value.

Mounted at /content/drive

```
<ipython-input-3-e82dd8e957b0>:5: DtypeWarning: Columns (8,9,10,11,12) have mixed types.
sales_data = pd.read_csv('/content/drive/My Drive/housing/housing-sales-borough.csv')
```

	Serial Number	List Year	Date Recorded	Town	Address	
0	2020177	2020	04/14/2021	Ansonia	323 BEAVER ST	
1	2020225	2020	05/26/2021	Ansonia	152 JACKSON ST	
2	2020348	2020	09/13/2021	Ansonia	230 WAKELEE AVE	
3	2020090	2020	12/14/2020	Ansonia	57 PLATT ST	
4	200500	2020	09-07-2021	Avon	245 NEW ROAD	

	Assessed Value	Sale Amount	Sales Ratio	Property Type	Residential Type	
0	133000	248400.0	0.5354	Residential	Single Family	
1	110500	239900.0	0.4606	Residential	Three Family	
2	150500	325000.0	0.4630	Commercial	NaN	
3	127400	202500.0	0.6291	Residential	Two Family	
4	217640	400000.0	0.5441	Residential	Single Family	

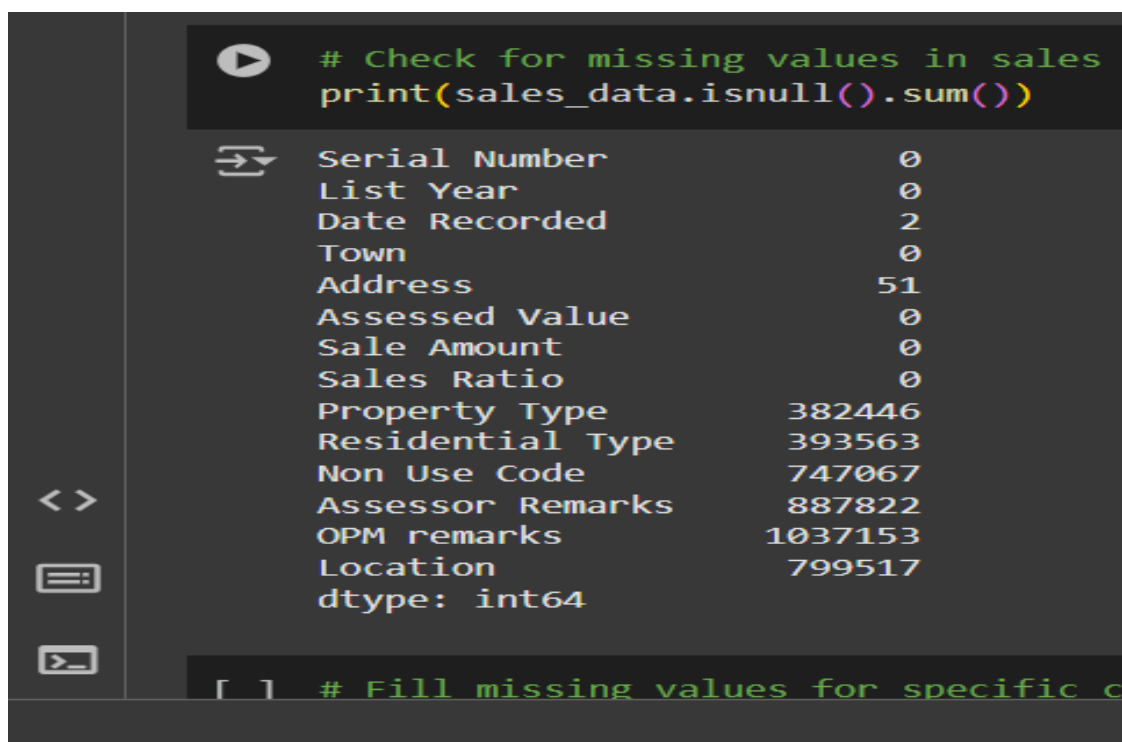
	Non Use Code	Assessor Remarks	OPM remarks	Location
0	NaN	NaN	NaN	POINT (-73.06822 41.35014)
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN

(Figure 1)

4.3 Dataset Overview for the sales data

This dataset contains 1,048,575 over a million records. it provides detailed information about property characteristics, financial assessments and transaction details making it a valuable resource for analyzing and predicting housing prices. Key numerical attributes include Sale Amount, Assessed Value, and Sales Ratio, while categorical features such as Property Type and Residential Type provide context about the type and use of the properties.

- **Missing Values** First I started with missing values and I found 7 columns with it.
- Property Type: 382,446
 - Residential Type: 393,563
 - Non Use Code: 747,067
 - Columns like Assessor Remarks (887,822 missing values), OPM Remarks (1,037,153 missing values), and Location (799,517 missing values) were seen as these have a lot of missing data and address with just 51. (Fig 2)



```
# Check for missing values in sales
print(sales_data.isnull().sum())
```

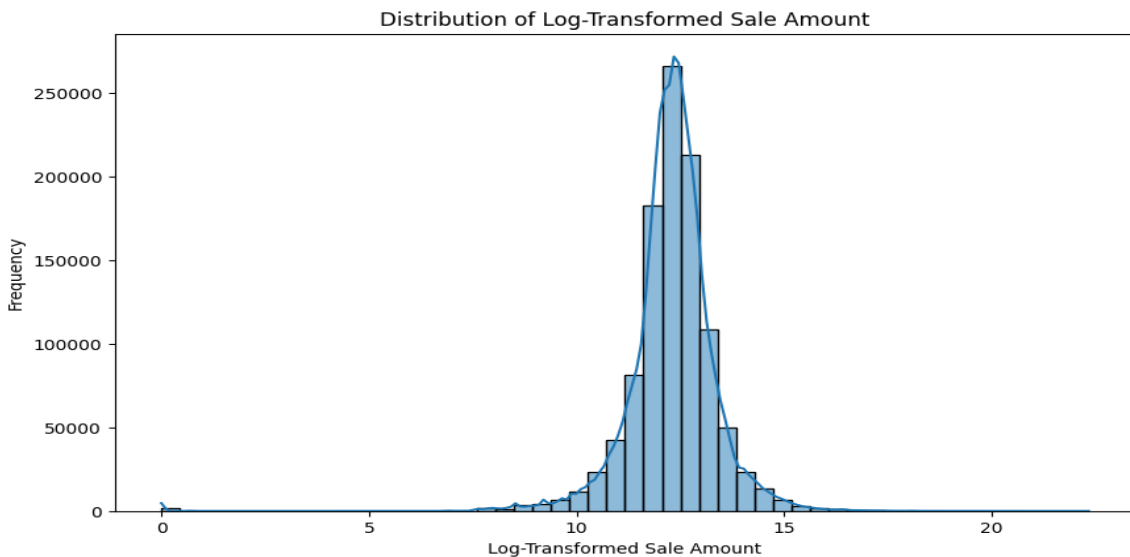
Serial Number	0
List Year	0
Date Recorded	2
Town	0
Address	51
Assessed Value	0
Sale Amount	0
Sales Ratio	0
Property Type	382446
Residential Type	393563
Non Use Code	747067
Assessor Remarks	887822
OPM remarks	1037153
Location	799517
dtype: int64	

```
[ ] # Fill missing values for specific c
```

(Figure 2)

➤ Skewness in Sale Amount

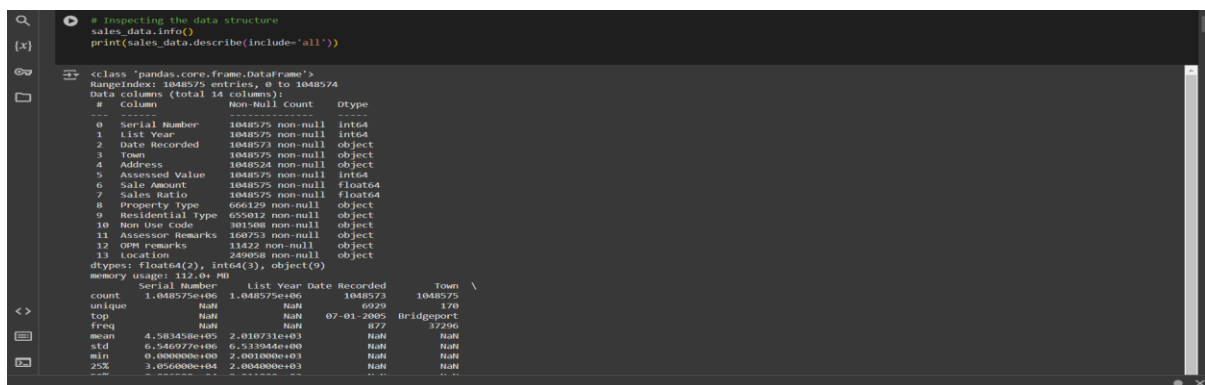
The sale amount column was highly skewed with most transactions clustering at lower values and a long tail for high value. (Fig 3)



(Figure 3)

➤ Irrelevant column

Columns such as serial number, list year, date recorded, and address were not relevant because they did not help in the model prediction, and I removed them during the pre-processing.

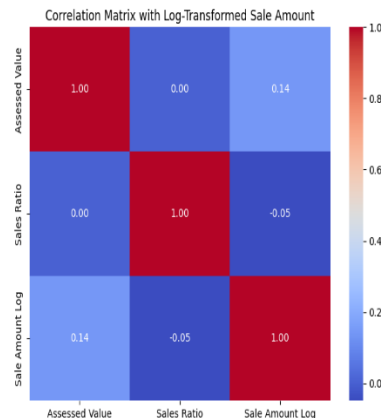


(Figure 4)

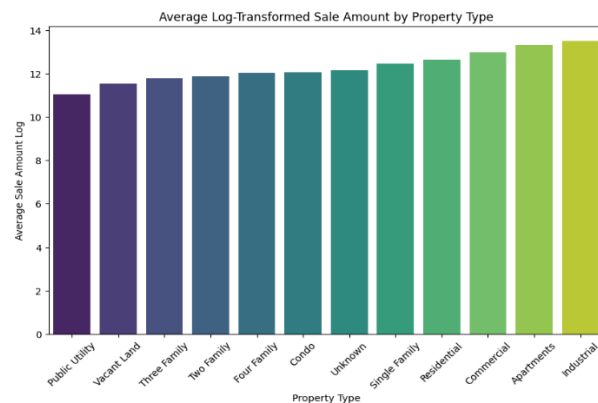
➤ Analysis of Numerical and Categorical Features.

The numerical features, like Sale Amount, Assessed Value, and Sales Ratio, showed key patterns and challenges. The Sale Amount distribution was highly skewed, and required a log transformation to normalize it, resulting in the feature Sale Amount

Log. This transformation produced a more well-shaped distribution suitable for analysis. Assessed Value demonstrated a strong positive correlation with log-transformed Sale Amount, highlighting its importance for housing prices (fig 5), while Sales Ratio shows some values, telling us that some potential anomalies require handling during preprocessing. Among categorical features, Property Type showed that Industrial and Apartments had the highest average Sale Amount Log, while Public Utility and Vacant Land had the lowest. Similarly, Residential Type was highest for single-family properties.



(Figure 5)



(Figure 6)

4.4 PRE-PROCESSING

It is an important part of any type of modelling because in this we prepare the dataset and ensure the quality and compatibility. With machine learning and deep learning models in this, I focused on handling missing values outliers, reducing skewness, encoding categorical variables and scaling numerical features. The dataset I got after this provided me with good results and a strong foundation for good analysis and predictive modelling.

- ✓ **Handling missing values** in The **Connecticut Real Estate Sales Data**, there were a lot of missing values across a lot of columns. To address this issue categorical attributes like Property Type, Residential Type, and Non Use Code were imputed with the value "Unknown to make them useful for the modelling. Whereas some columns have excessive missing data Assessor Remarks with 887,822 missing values), OPM Remarks 1,037,153 missing, and Location 799,517 missing were removed entirely because they added only a small amount of value to the analysis. With this method, a lot of data was cleaned by reducing noise and redundancy. (Fig 2)
- ✓ **Outlier detection and handling** in the sales datasets outliers were seen and handled properly to prevent them from skewing the results. A combination of **percentile capping** 1st and 99th percentiles and **IQR-based capping** values outside $1.5 \times \text{IQR}$ was applied to numerical features such as Assessed Value, Sales Ratio, and

Sale Amount Log. Outliners were also removed entirely. This process reduces the influence of extra values while maintaining the data. (Fig 7 and Fig 9)

Percentile Capping applied.

	Assessed Value	Sales Ratio	Sale Amount Log	Town Encoded \
count	1.048575e+06	1.048575e+06	1.048575e+06	1.048575e+06
mean	2.223543e+05	7.995475e-01	1.231438e+01	1.229900e+01
std	3.076911e+05	9.599892e-01	9.320277e-01	5.340584e-01
min	2.500000e+03	9.544096e-03	9.210440e+00	1.149000e+01
25%	8.841000e+04	4.818820e-01	1.186359e+01	1.193866e+01
50%	1.395500e+05	6.171429e-01	1.234584e+01	1.219634e+01
75%	2.267300e+05	7.775000e-01	1.282126e+01	1.254305e+01
max	2.226718e+06	8.308520e+00	1.488022e+01	1.408139e+01

	Property Type Commercial	Property Type Condo \
count	1048575.0	1.048575e+06
mean	0.0	1.005364e-01
std	0.0	3.007141e-01
min	0.0	0.000000e+00
25%	0.0	0.000000e+00
50%	0.0	0.000000e+00
75%	0.0	0.000000e+00
max	0.0	1.000000e+00

	Property Type Four Family	Property Type Industrial \
count	1048575.0	1048575.0
mean	0.0	0.0
std	0.0	0.0
min	0.0	0.0
25%	0.0	0.0
50%	0.0	0.0
75%	0.0	0.0
max	0.0	0.0

	Property Type Public Utility	Property Type Residential ... \
count	1048575.0	1.048575e+06
mean	0.0	1.018868e-01
std	0.0	3.024996e-01

(Figure 7)

Skewed Columns:

Assessed Value	4.399101
Sales Ratio	6.032529
Town Encoded	1.243072
Property Type Condo	2.656772
Property Type Residential	2.632162
Property Type Three Family	8.962432
Property Type Two Family	6.060747
Property Type Unknown	0.562043
Residential Type Three Family	8.058011
Residential Type Two Family	5.418026
Residential Type Unknown	0.514939
Non Use Code 07 - Change in Property	5.015017
Non Use Code 08 - Part Interest	7.766147
Non Use Code 14 - Foreclosure	4.081970
Non Use Code 25 - Other	3.688776
Non Use Code 7	8.316584
Non Use Code Unknown	-0.938808

dtype: float64

```
[ ] import numpy as np

# List of numeric, positive, highly skewed columns
skewed_numeric_cols = ['Assessed Value', 'Sales Ratio']

# Apply log1p to handle skewness
for col in skewed_numeric_cols:
    sales_data[col] = np.log1p(sales_data[col])

print("Log transformation applied to:", skewed_numeric_cols)
```

Log transformation applied to: ['Assessed Value', 'Sales Ratio']

(Figure 8)

Skewness after log transformation:

Assessed Value	-0.594987
Sales Ratio	3.135419

dtype: float64

```
# Apply capping to handle outliers
columns_to_cap = ['Assessed Value', 'Sales Ratio', 'Sale Amount Log']

for col in columns_to_cap:
    Q1 = sales_data[col].quantile(0.25)
    Q3 = sales_data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    sales_data[col] = sales_data[col].clip(lower=lower_bound, upper=upper_bound)

print("Outliers capped for:", columns_to_cap)
```

Outliers capped for: ['Assessed Value', 'Sales Ratio', 'Sale Amount Log']

```
# Apply outlier removal
columns_to_check = ['Assessed Value', 'Sales Ratio', 'Sale Amount Log']

for col in columns_to_check:
    Q1 = sales_data[col].quantile(0.25)
    Q3 = sales_data[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
```

(Figure 9)

- ✓ **Skewness Reduction** in the sales data there were a lot of numerical features that has skewness, which could make the model performance bad. To handle this I used logarithmic transformation `np.log1p()` to Assessed Value, Sales Ratio, and Sale Amount. Before applying, Assessed Value and Sales Ratio had skewness values of 4.39 and 6.03. After the log transformation, their skewness was reduced to -0.59 and 3.13, resulting in more normalized distributions. (Fig 8 and Fig 9)

- ✓ **Categorical Encoding** was used in the sales data Categorical variables were converted into numeric values to enable models to learn properly. The Town column was target-encoded by mapping each town to the mean value of the Sale Amount Log. one-hot encoding was also used for categorical features such as Property Type, Resi-

dential Type, and Non Use Code, giving binary columns for each category. After encoding, r columns like the original Town and Sale Amount are replaced by Sale Amount Log. This step expanded the dataset to 126 columns including the encoded features. (Fig 10 and Fig 11)

```
Applied one-hot encoding to 'Non Use Code'.
Converted boolean columns to numeric.
All columns are numeric. Dataset is ready for modeling!
Final Sales Dataset Shape: (1048575, 126)
```

	Assessed Value	Sales Ratio	Sale Amount Log	Town Encoded \
0	133000	0.5354	12.422800	12.054082
1	110500	0.4606	12.387982	12.054082
2	150500	0.4630	12.691584	12.054082
3	127400	0.6291	12.218500	12.054082
4	217640	0.5441	12.899222	12.767125

	Property Type_Commercial	Property Type_Condo	Property Type_Four Family	\
0	0	0	0	
1	0	0	0	
2	1	0	0	
3	0	0	0	
4	0	0	0	

	Property Type_Industrial	Property Type_Public Utility	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	Property Type_Residential	...	Non Use Code_73_0	Non Use Code_74	\
0	1	...	0	0	
1	1	...	0	0	
2	0	...	0	0	
3	1	...	0	0	
4	1	...	0	0	

(Figure 10)

	Town Encoded	Property Type_Commercial	Property Type_Condo	\
0	12.054082	False	False	
1	12.054082	False	False	
2	12.054082	True	False	
3	12.054082	False	False	
4	12.767125	False	False	

	Property Type_Four Family	Property Type_Industrial	...	\
0	False	False	False	
1	False	False	False	
2	False	False	False	
3	False	False	False	
4	False	False	False	

	Property Type_Single Family	Property Type_Three Family	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	

	Property Type_Two Family	Property Type_Unknown	Property Type_Vacant Land	\
0	False	False	False	
1	False	False	False	
2	False	False	False	
3	False	False	False	
4	False	False	False	

	Residential Type_Four Family	Residential Type_Single Family	\
0	False	True	
1	False	False	
2	False	False	
3	False	False	
4	False	True	

	Residential Type_Three Family	Residential Type_Two Family	\
0	False	False	
1	False	False	
2	False	False	
3	False	False	
4	False	False	

(Figure 11)

- ✓ **Feature Scaling** was used in the sales data to maintain consistent scales across numerical features, and RobustScaler was applied to the Assessed Value, Sales Ratio, and Sale Amount Log. This method effectively mitigated the impact of remaining outliers and standardized the features for machine learning algorithms. Scaling was particularly important for ensuring that features with larger magnitudes did not dominate the modelling process. (Fig 12)

```
Robust Scaling applied.
```

	Assessed Value	Sales Ratio	Sale Amount Log	Town Encoded \
count	1.048575e+06	1.048575e+06	1.048575e+06	1.048575e+06
mean	1.660891e-02	8.302213e-02	-1.816449e-02	1.698511e-01
std	8.744670e-01	9.104340e-01	8.760591e-01	8.836364e-01
min	-1.984656e+00	-1.980211e+00	-2.003565e+00	-1.168694e+00
25%	-4.846563e-01	-4.802112e-01	-5.035646e-01	-4.263442e-01
50%	0.000000e+00	0.000000e+00	0.000000e+00	0.000000e+00
75%	5.153437e-01	5.197888e-01	4.964354e-01	5.736558e-01
max	2.015344e+00	2.019789e+00	1.996435e+00	3.118945e+00

	Property Type_Commercial	Property Type_Condo	\
count	1048575.0	1.048575e+06	
mean	0.0	1.005364e-01	
std	0.0	3.007141e-01	
min	0.0	0.000000e+00	
25%	0.0	0.000000e+00	
50%	0.0	0.000000e+00	
75%	0.0	0.000000e+00	
max	0.0	1.000000e+00	

	Property Type_Four Family	Property Type_Industrial	\
count	1048575.0	1048575.0	
mean	0.0	0.0	
std	0.0	0.0	
min	0.0	0.0	
25%	0.0	0.0	
50%	0.0	0.0	
75%	0.0	0.0	
max	0.0	0.0	

	Property Type_Public Utility	Property Type_Residential	...	\
count	1048575.0	1.048575e+06	...	
mean	0.0	1.018868e-01	...	
std	0.0	3.024996e-01	...	
min	0.0	0.000000e+00	...	
25%	0.0	0.000000e+00	...	

(Figure 12)

- ✓ **Removal of Low-Variance Features** in the sales data the Low-Variance columns were identified and removed as they contributed little to the prediction of the dataset. After preprocessing, no significant low-variance features were left, allowing the model to focus on meaningful and varying features.

- ✓ **Removal of Constant** To make the dataset more processable and to improve its efficiency, constant columns with a single unique value like Property Type Commercial were removed. Also, redundant features like Property Type Single Family and Unknown were dropped. By doing these steps its makes the data less noisy.

- **Preprocessing for the USA housing listing**

- ✓ **Handling Missing Values** for the housing dataset, the categorical columns laundry options and parking options were imputed using their mode most frequent value. For lat, long, and description, rows with missing values were dropped due to their less importance for the analysis. After handling missing data, the dataset size was reduced to 383,057 rows. And there was no missing value. (Fig 14)

```
{x} dtype: int64
Shape after handling missing values: (383057, 22)

# Select the columns for one-hot encoding
columns_to_encode = ['region', 'type', 'laundry_options', 'parking_options', 'state']

# Apply one-hot encoding
rental_data_encoded = pd.get_dummies(rental_data, columns=columns_to_encode, drop_first=True)

# Check the columns after encoding
print("Columns after One-Hot Encoding:")
print(rental_data_encoded.columns)

# Check the shape after encoding
print("Shape of dataset after One-Hot Encoding:", rental_data_encoded.shape)

Columns after One-Hot Encoding:
Index(['id', 'url', 'region_url', 'price', 'sqfeet', 'beds', 'baths',
       'cats_allowed', 'dogs_allowed', 'smoking_allowed',
       ...,
       'state_sd', 'state_tn', 'state_tx', 'state_ut', 'state_va', 'state_vt',
       'state_wa', 'state_wi', 'state_wv', 'state_wy'],
      dtype='object', length=491)
Shape of dataset after One-Hot Encoding: (383057, 491)
```

(Figure 13)

```
{x} # Check missing values in the hosing dat
print("Missing values in each column:")
print(rental_data.isnull().sum())

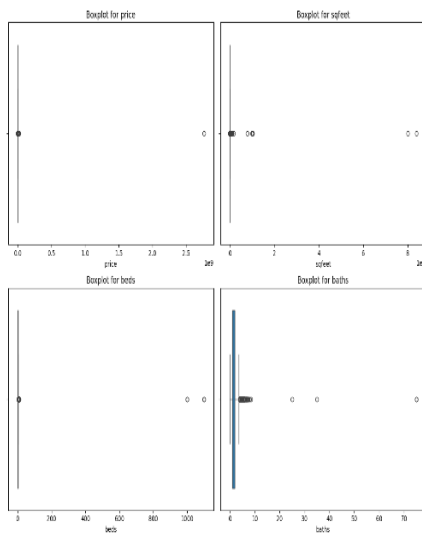
Missing values in each column:
id                0
url               0
region           0
region_url       0
price            0
type             0
sqfeet           0
beds             0
baths            0
cats_allowed     0
dogs_allowed     0
smoking_allowed  0
wheelchair_access 0
electric_vehicle_charge 0
comes_furnished  0
laundry_options  79026
parking_options  140687
image_url        0
description       2
lat              1918
long             1918
state            0
dtype: int64

[ ] # Check data types of each column
print("Data types of each column:")
```

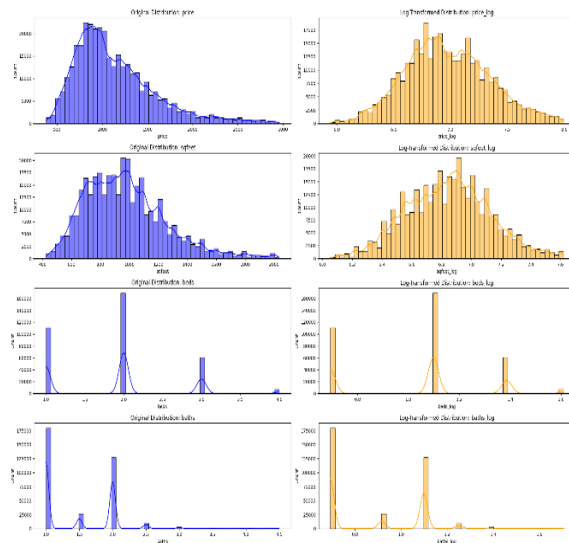
(Figure 14)

- ✓ **Dropping Irrelevant Columns** in the housing data columns like id, url, region url, image url and description were dropped from the dataset. because they contained no meaningful information, dropping these columns reduced the redundancy and focused the data on important features. (Fig 13)

- ✓ **Encoding Categorical Variables** for the housing data one-hot encoding was used for the columns like region, type, laundry options, parking options, and state. This process converted these features into binary variables, enabling them to be used as inputs for numerical algorithms. (Fig 14)
- ✓ **Log transformation** of the skewness in numeric features like price, sqft, beds, and baths a log transformation was used. This transformation normalized the data and distributions of these variables, making them more symmetric and reducing the extreme values. (Fig 15 and Fig 16)

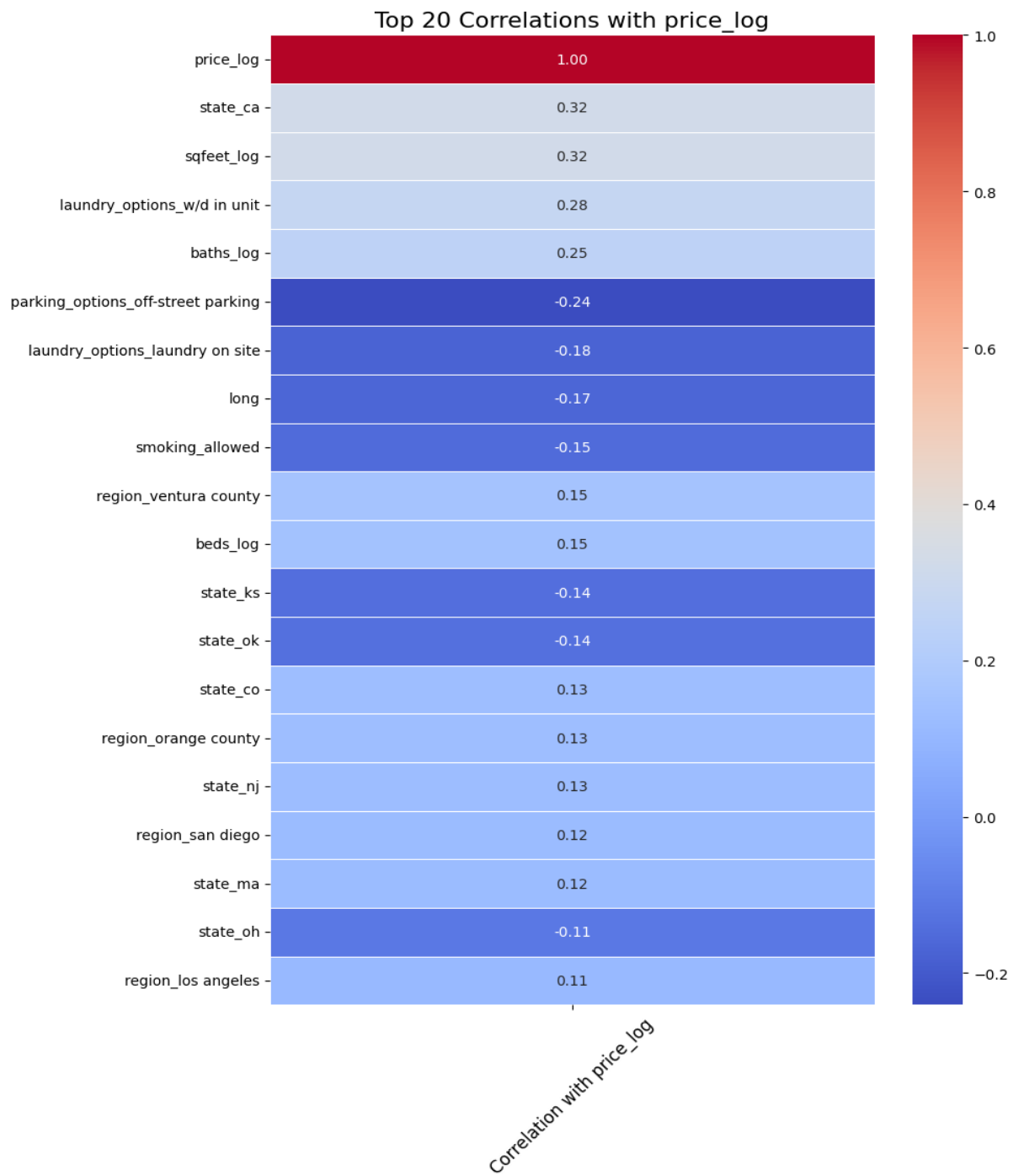


(Figure 15)



(Figure 16)

- ✓ **Outlier Detection and Removal** in the housing data the outliers were handled by the combination of the Interquartile Range (IQR) method and Z-Score. The IQR method calculated the upper and lower bounds for each feature, while the Z-Score method identified extreme values based on standard deviations. The outliers were still there, and they were handled by combining the stricter bounds from both methods.
- ✓ **Correlation Analysis:** I performed a correlation analysis to check the relationship between features and the target variable, the price log. Features such as sqft log 0.32 and baths log 0.25 showed the strongest positive correlations with price log. Some categorical variables, such as state ca 0.32 also show correlations. (Fig 17)
- ✓ **Scaling** was applied to standardize the data. Both Standard Scaling and Min-Max Scaling were used to normalize the features before deep learning model. Standard Scaling centered the data around a mean of 0, while Min-Max Scaling converted the data to a fixed range of 0 and 1. Then these were used for both training and testing splits.



(Figure 17)

4.5 Data Split

After doing all the preprocessing, which I explained in the dataset section, I split the dataset into testing and training sets to prepare it for modelling. The target variable for the Connecticut Real Estate Sales Data was the sales amount log, which was separated during the feature set. An 80 – 20 per cent split was done using the train test split function from sklearn. By doing this, 80 per cent of the data was used for training, and 20 was going to be used for testing. The same I did for the USA Housing Listings Data; the target variable price_log was separated during the feature set, and the same 80-20 ratio was applied to separate the data. I set the random state parameter to 42 for both datasets to ensure its consistency.

4.6 LinearRegression

I used this model as the baseline model to predict housing sales and rental prices. I chose this because it is simple to implement and gives a good starting point for checking the performance of more advanced models. This model is based on a linear relationship between the features and the target variable, which limits its capacity to capture complex patterns. It is a good model for a basic benchmark for comparison. The purpose of using this is to check how the dataset gives results using the simple and straightforward approach. I used the LinearRegression class from Python's sklearn library. Then I trained it on X train and y train and tested it on the test data X test and y test. During the evaluation process, I used three different metrics: Mean Absolute Error (MAE) to understand the average size of prediction errors, Root Mean Squared Error (RMSE) to check large errors, and R² Score to measure how well the model explains the variance in the target variable.

4.7 Random Forest Model

I used this for the USA Housing Listings Data. It also combines multiple decision trees by group learning method to give robust and accurate predictions. I chose this model because of its ability to handle non-linear relationships and its flexibility against overfitting also its efficiency in handling datasets with diverse feature types. It also provides feature importance, which gives insights into which attributes contribute most to the predictions. I used this model using the RandomForestRegressor class from the sklearn library. Then I configured it with 100 decision trees n_estimators=100 and with the fixed random state random_state=42 to ensure data consistent and reproducible results. I trained the model on the preprocessed training dataset rental X train and rental y train and evaluated it on the test dataset rental X test and rental y test. Then I measured the Predictions using three metrics: MAE to evaluate average prediction errors, RMSE to check for large errors, and R² Score to measure how well the model explained the variance in rental price.

4.8 LightGBM Model

I used Light Gradient Boosting Machine model on the Connecticut Real Estate Sales Data. Because of its highly efficient gradient-boosting framework that helps it increase speed and accuracy. I chose this model because of its ability to handle large datasets, capture complex patterns, and optimise performance through its advanced techniques, like histogram-based decision tree learning. Also, its capacity to handle numerical and categorical features.

I implemented this model using Python's lightgbm library. The training and testing datasets were prepared using the lgb. I used sales X train and sales y train for training, and sales X test and sales y test were used for testing. The model was configured objective regression for predicting continuous values, metric rmse to check performance, and boosting type gbdt for gradient boosting. learning rate 0.1 to control weight updates, num leaves 31 for tree complexity and random state 42 for reproducibility. It was trained using the lgb.train() function with a maximum of 1,000 boosting iterations and early stopping was enabled after 50 rounds to avoid overfitting. To check the model performance I used MAE, RMSE and R2 score.

4.9 Artificial Neural Network (ANN)

I used ANN, a type of deep learning model that was made for the regression task. These models were made to copy the way the human brain processes information, using interconnected layers of artificial neurons to model relationships in data I chose ANN because its power to handle nonlinear relationships which old models might not capture. Using multiple hidden layers with activation functions and regularisation techniques was designed to learn intricate patterns in the dataset and generalise unseen data well.

- **Model Architecture for the sales Data**
 - ✓ I used TensorFlow Keras sequential API.
 - ✓ Input layer the scaled input features sales X train scaled, with dimensions equal to the number of features in the dataset.
 - ✓ Hidden Layer of 128 neurons with the ReLU Rectified Linear Unit activation function to introduce non-linearity. The Dropout layer 0.3 rate to reduce overfitting.
 - ✓ Second Hidden Layer of 64 neurons with ReLU activation and a Dropout layer 0.2 rate to provide additional regularisation. Third Hidden Layer: 32 neurons with ReLU activation
 - ✓ Output Layer with single neuron with a linear activation function to predict the continuous target variable Sale Amount Log.
- **Model Compilation**
 - ✓ Optimizer: Adam optimiser with a learning rate of 0.001, selected because of its efficient adaptive learning rate mechanism.
 - ✓ Loss Function: Mean Squared Error (MSE), ideal for regression tasks as it penalizes larger errors more heavily.

- ✓ Evaluation Metric: Mean Absolute Error (MAE) was used alongside MSE to provide an intuitive understanding of average prediction errors.
- **Training Process**
 - ✓ I set the batch size to 16 for controlling the number of samples processed before updating the model's weights.
 - ✓ 50 epochs were used to allow sufficient iterations for learning without excessive training.
 - ✓ Early Stopping callback is also used to monitor validation loss and stop training after 10 consecutive epochs without improvement. This helped the model to not overfit, and the best weights were restored.
- **Model Architecture for the USA housing Dataset**
 - ✓ The scaled feature set rental X train scaled serves as input with dimensions matching the number of features.
 - ✓ **Hidden Layers**
 - Three dense layers were used with 256, 128, and 64 neurons
 - I also used ReLU Rectified Linear Unit activation function check the complex patterns .
 - Batch Normalization layers were applied after each dense layer to stabilize and accelerate training.
 - Dropout layers with rates of 0.3 and 0.2 were used to randomly deactivate neurons during training, reducing overfitting.
 - ✓ I used a single neuron with no activation function to predict the target variable price Log as the output layer.
- **Model Compilation**
 - ✓ Adam optimizer was used with a learning rate of 0.001, because of its adaptive learning capabilities.
 - ✓ Loss Function: Mean Squared Error MSE, suitable for regression tasks as it penalizes larger errors more heavily.
 - ✓ Mean Absolute Error MAE provides a intuitive measure of average prediction error.
- **Training Process**
 - ✓ I trained this model on the scaled training dataset rental x train scaled and rental y train 20 % of the data was kept for the validation to monoter the model performance during the training
 - ✓ I used the batch size of 64 to process a subset of the data at each training step to maintain both memory and computation time
 - ✓ I trained the model till 100 epochs
 - ✓ Callbacks like early stopping and ReduceLROnPlateau I reduced the learning rate by 0.5 if validation loss plateaued for 5 epochs, improving convergence and val loss and halted training after 15 consecutive epochs without improvement, ensuring the best weights were restored

- **Evaluation**

- ✓ Mean Absolute Error (MAE): Measures the average magnitude of prediction errors.
- ✓ Root Mean Squared Error (RMSE): Captures the influence of larger errors.
- ✓ R^2 Score: Quantifies how well the model explains the variance in both datasets.

4.10 Challenges

While I was doing the preprocessing, I had many problems, like missing values in the sales dataset. Where columns like assessor remarks and OPM remarks had a lot of missing data. I addressed this by removing the columns with high missing values and imputing categorical attributes. Another big issue was dealing with outliers in both datasets. I used techniques like percentile capping and IQR and Z score-based methods. Also, skewed distribution in features like sale amount and price was another issue, which I solved with log transformation. In modelling, overfitting was the biggest challenge, which I fixed with dropout layers, early stopping and dynamic learning rate.

Results

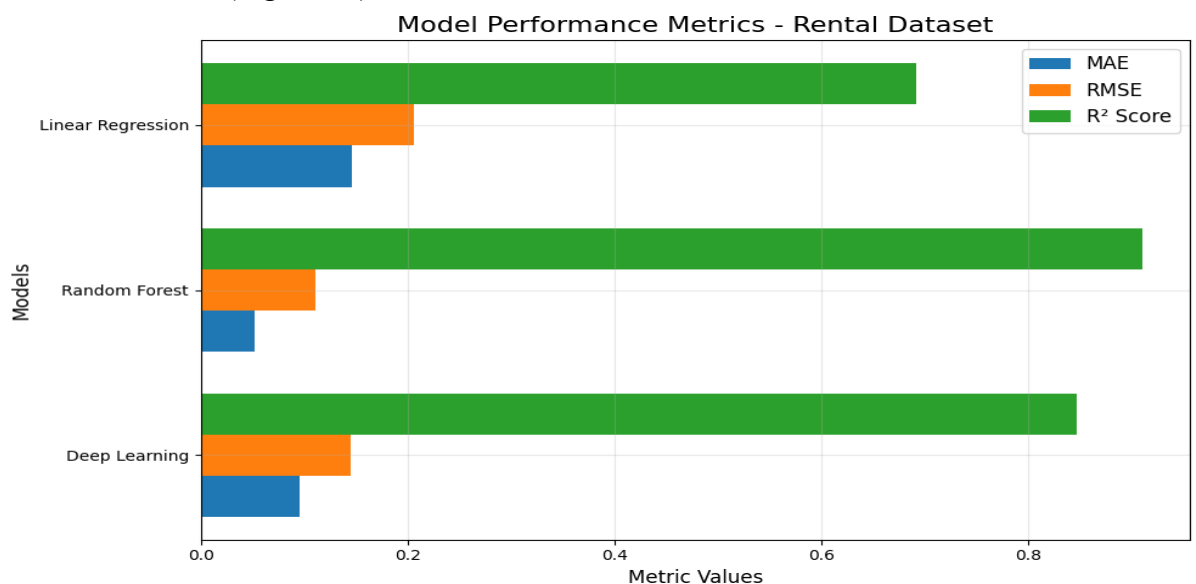
5. Model prediction

I checked and compared the model prediction by visualising the predicted and actual price values. With real-world data. for both datasets. Metrics such as Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score were used to evaluate model effectiveness

5.1 USA Housing Listing Data Prediction

Linear Regression I used this as the baseline model to provide a benchmark for rental price prediction.

- **MAE:** 0.1467
- **RMSE:** 0.2062
- **R^2 Score:** 0.6919
- it showed me an understanding that a more advanced model is necessary to capture the data nuances. (Figure 18)



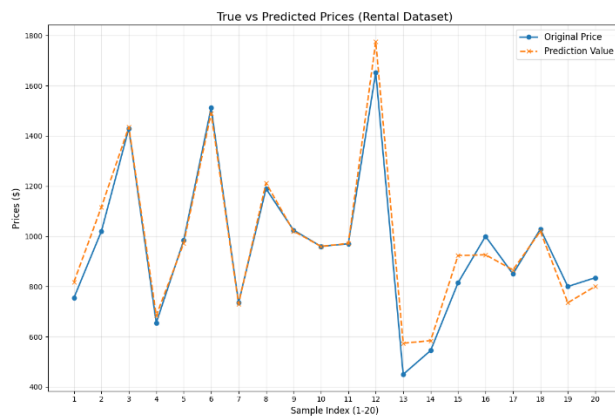
(Figure 18)

Random forest outperformed the linear regression with a greater gap between the metrics. Showing the nonlinear relationship between the features.

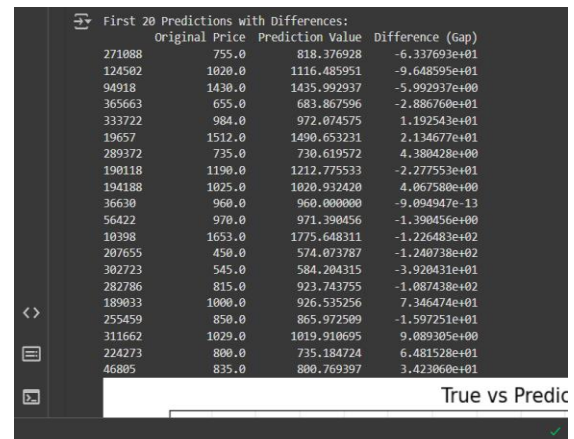
- **MAE:** 0.0524
- **RMSE:** 0.1108
- **R^2 Score:** 0.9111

I predicted the first 20 predictions for the true vs predicted prices. I did the log transformation to convert it back to the original scale with the function `np.expml`. for the error analysis I also checked the difference between original and predicted value.

I also plotted a line plot for the true value and predicted value, and most of them aligned properly. (Figures 19 and 20)



(Figure 19)

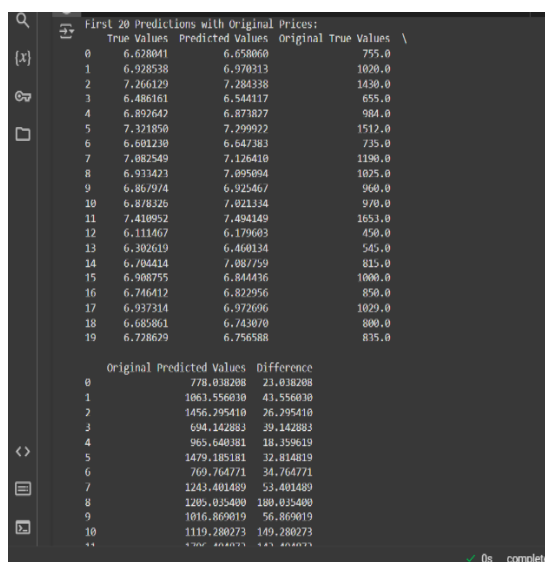


(Figure 20)

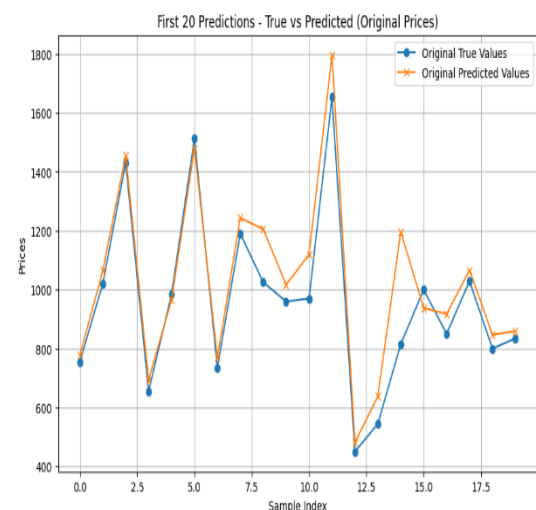
Deep learning Model (ANN)

I used batch normalisation and multiple hidden layers to capture the complex relationship in the data, but it performed lower than the random forest.

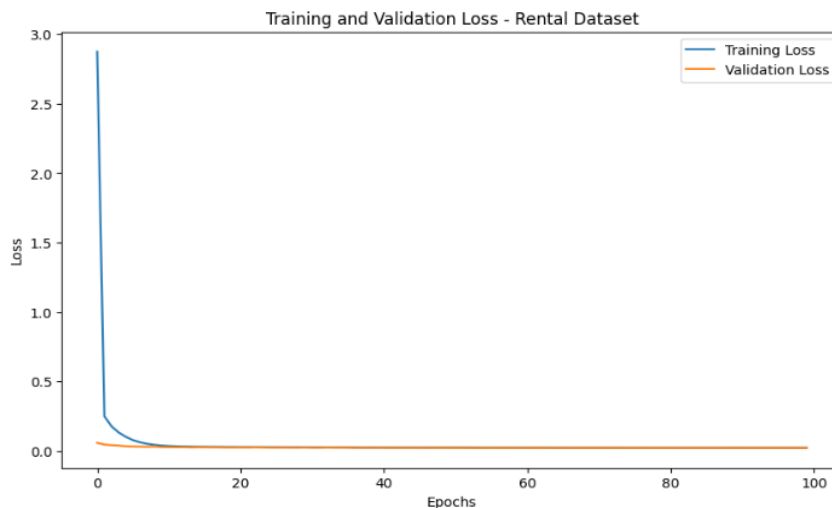
- **MAE:** 0.0951
- **RMSE:** 0.1451
- **R² Score:** 0.8474
- The validation loss curve showed stable training. I also trained it for the first 20 indexes and converted it back into the original price. (Figure 21,22 and 23)



(Figure 21)



(Figure 22)



(Figure 23)

5.2 Connecticut Real Estate Sales Data Prediction

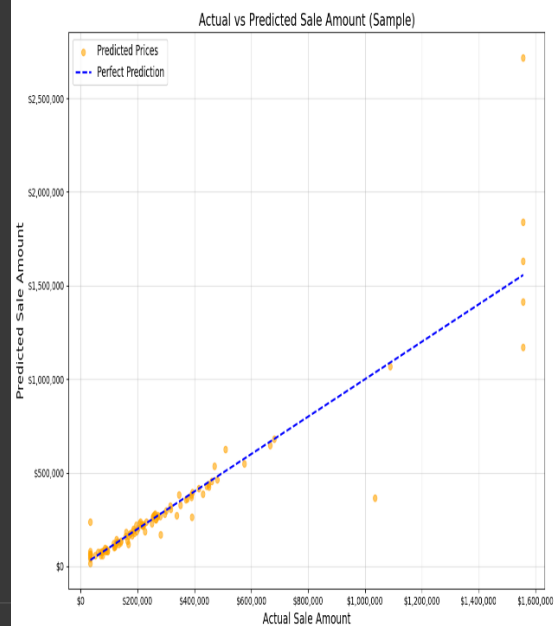
The **Linear Regression** model was used as the baseline to predict the Sale Amount Log.

- **MAE:** 0.1300
- **RMSE:** 0.2811
- **R² Score:** 0.8881

I did the reverse transform to the original value and predicted the 20 values, showing very good model accuracy. The maximum of the predictions was aligned with the actual price. (Figure 24 and 25)

	Actual Price (Dollars)	Predicted Price (Dollars)	
781974	330000.0	331586.150060	
937737	230000.0	228522.802286	
907828	480000.0	399974.492124	
784628	201000.0	202614.177405	
662460	140000.0	139690.605290	
280139	60000.0	58533.026578	
355572	98000.0	107451.671768	
749979	230000.0	228789.365860	
374753	140000.0	140571.117978	
17327	255000.0	253550.376297	
658472	125000.0	68713.484196	
700377	160000.0	159899.366587	
880383	580000.0	571814.146290	
677181	179000.0	178288.386727	
18941	370000.0	368136.387782	
735693	507500.0	510059.211335	
1034300	480000.0	479748.856469	
389058	122000.0	122028.221175	
473202	170000.0	170348.965089	
677891	139900.0	141034.686085	
Difference (Dollars)			
781974		-1586.150060	
937737		1477.197714	
907828		25.507876	
784628		-1614.177405	
662460		309.394710	
280139		1466.973422	
355572		-9451.671768	
749979		1210.634140	
374753		-571.117978	
17327		1449.623703	
658472		56286.515804	

(Figure 24)



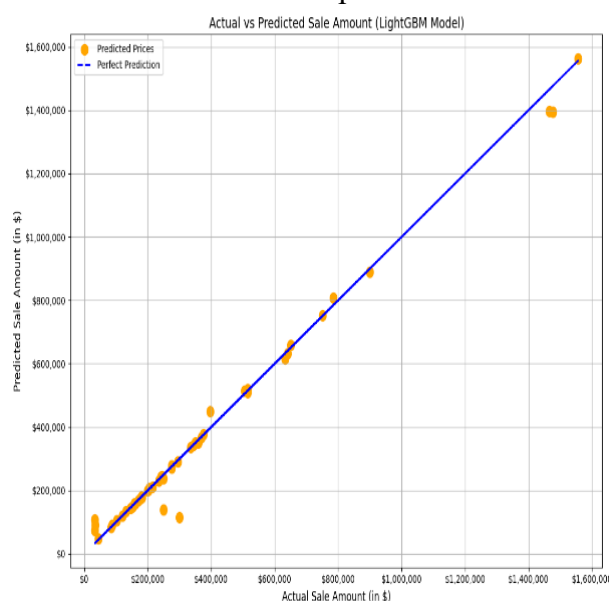
(Figure 25)

LightGBM Model

The LightGBM model showed the highest accuracy. It handled the dataset's non-linear relationships and feature interactions. (Figures 26 and 27)

- **MAE:** 0.0484
- **RMSE:** 0.1717
- **R² Score:** 0.9583

The reverse-transformed predictions closely matched the actual prices, with minimal errors even in high-value properties. This makes LightGBM the best-performing model for the sales dataset. And the predictions were maximum aligned to the actual price.



(Figure 26)

Top 20 Predictions vs Actual Values :

	Actual Price (Dollars)	Predicted Price (Dollars)	Difference (Dollars)
0	330000.0	331586.150060	-1586.150060
1	230000.0	228522.802286	1477.197714
2	400000.0	399974.492124	25.507876
3	201000.0	202614.177405	-1614.177405
4	140000.0	139690.605290	309.394710
5	60000.0	58533.026578	1466.973422
6	90000.0	107451.671768	-9451.671768
7	230000.0	228789.365868	1210.634140
8	140000.0	140571.117978	-571.117978
9	255000.0	253550.376297	1449.623703
10	125000.0	68713.484196	56286.515804
11	160000.0	159899.366587	100.633413
12	500000.0	571814.146290	8185.853710
13	179000.0	178288.386727	711.613273
14	370000.0	368136.387782	1863.612218
15	507500.0	510059.211335	-2559.211335
16	480000.0	479748.856469	251.143531
17	122000.0	122028.221175	-28.221175
18	170000.0	170348.965089	-348.965089
19	139900.0	141034.686085	-1134.686085

```
[ ] # Drop only the columns that exist in the dataset
redundant_features = [
    'Property.Type.Single.Family',
```

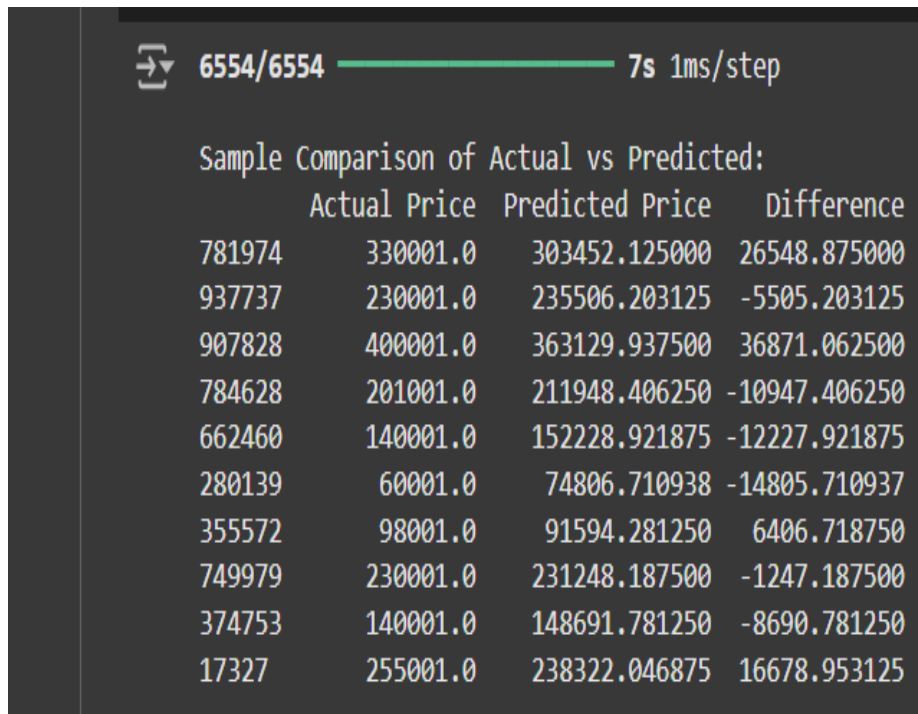
(Figure 27)

Deep Learning Model

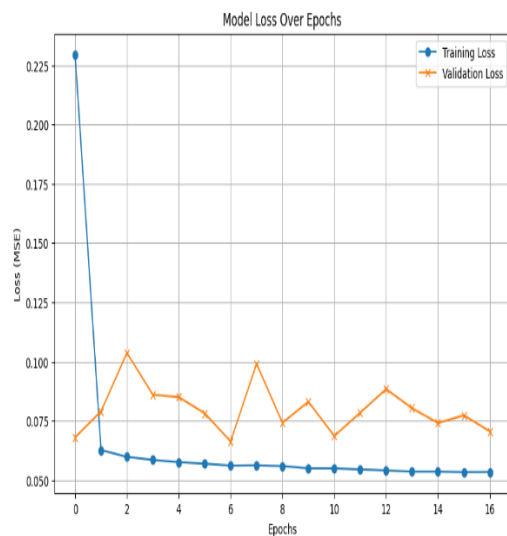
The Deep Learning model showed the lowest score compared to the lightGBM, even after having multiple layers. The maximum of the predictions were aligned properly

- **MAE:** 0.0664
- **RMSE:** 0.2576
- **R² Score:** 0.9061

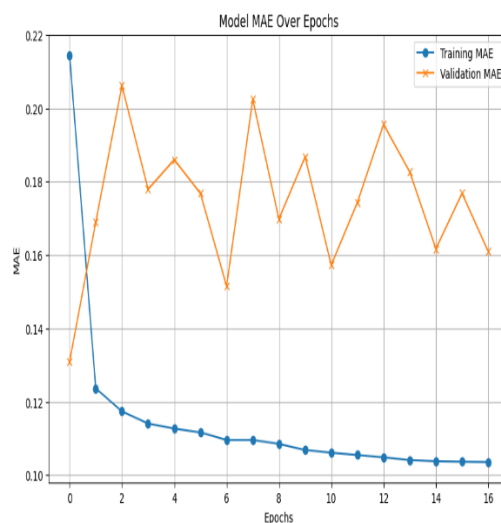
While its predictions were strong, occasional outliers and the small data slightly affected its overall performance. I made 10 predictions, including loss and MAE over epochs. (Figures 28, 29, 30, 31, 32)



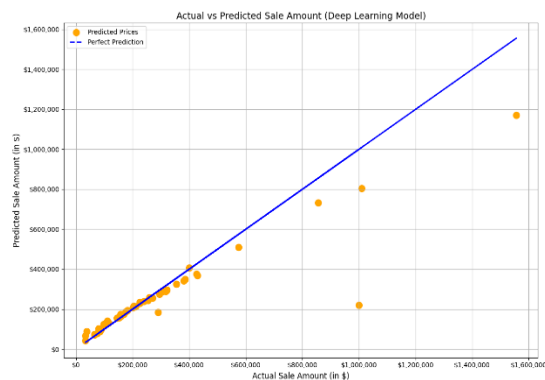
(Figure 28)



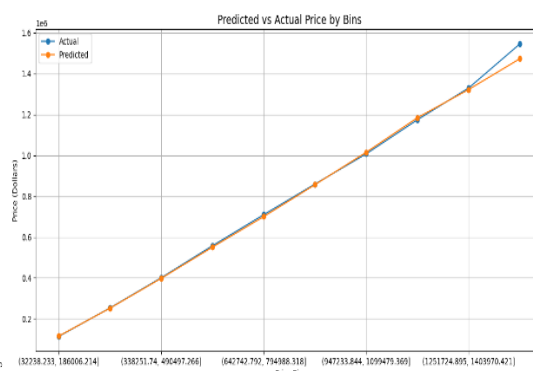
(Figure 29)



(Figure 30)



(Figure 31)



(Figure 32)

6. Analysis and Discussion

I used linear regression as a baseline model because of its less capacity in capturing non-linear relationships. Advanced models like the random forest, LightGBM and ANNs performed better than it. random forest showed robust prediction. While handling diverse features, it shows an R2 score of 0.9111 and RMSE of 0.1108 in rental price prediction. Because of the hyper-tuning, the LightGBM model has shown very good results with an R2 score of 0.9583 and RMSE of 0.1717. it showed that it worked with both categorical and numerical features, making it ideal for this application.

ANNs showed the ability to capture mixed relationships. For the sales dataset, it showed an R2 score of 0.9061 and an RMSE of 0.2576. It showed good predictive power but was lower than lighGBM. For rental data, the R2 score is 0.8992 and an RMSE of 0.1125. ANNs use a lot of resources and are sensitive to hyperparameter tuning, making them less powerful than LightGBM and Random Forest.

Preprocessing played an important role in the success of all models. Log transformations handle the maximum of the skewness in the target variable like sale amount and price, ensuring that the model can learn without being skewed by extreme values. Outliers removals stabilise the data that could make training bad. Feature scaling for ANN models was important for smooth optimisation. Handling missing values through imputation while dropping the irrelevant columns it showed direct link to my research question.

LightGBM showed higher accuracy in all of them, and it is also best for larger datasets with the best accuracy. It is particularly suited for large-scale property valuation systems and market analysis, making these the best tools in the real world. Where random forest offers robustness and ease of implementation, making it good for medium-sized data. ANNs, are best suited for tasks where computational resources are available, and capturing nuanced patterns is essential while not suited for the smaller dataset.

7. Conclusion

The LightGBM model is the best model among all of the other models because it handles the larger dataset and complex patterns. also Deep learning models also performed better than other models, showing its ability to learn intricate relationships while Linear Regression served as a useful baseline. These models can be applied to real-world tools for property valuation, helping users make informed decisions. However, further testing on diverse datasets and refining the models could improve their accuracy and reliability. Future work could involve exploring advanced techniques and integrating these systems into live applications for broader use.

References

- Lahmiri, S., Bekiros, S. and Avdoulas, C. (2023). A comparative assessment of machine learning methods for predicting housing prices using Bayesian optimization. *Decision Analytics Journal*, 6, p.100166.
<https://doi.org/10.1016/j.dajour.2023.100166>
- Adetunji, A.B., Akande, O.N., Ajala, F.A., Oyewo, O., Akande, Y.F. and Oluwadara, G. (2022). House Price Prediction using Random Forest Machine Learning Technique. *Procedia Computer Science*, 199, pp.806–813.
[doi:https://doi.org/10.1016/j.procs.2022.01.100](https://doi.org/10.1016/j.procs.2022.01.100).
- Selim, H. (2009). Determinants of house prices in Turkey: Hedonic regression versus artificial neural network. *Expert Systems with Applications*, 36(2), pp.2843–2852. doi:<https://doi.org/10.1016/j.eswa.2008.01.044>.
- Sharma, H., Harsora, H. and Ogunleye, B. (2024). An Optimal House Price Prediction Algorithm: XGBoost. *Analytics*, [online] 3(1), pp.30–45.
doi:<https://doi.org/10.3390/analytics3010003>.
- Wijaya, R. (2023). *Multi Level Dense Layer Neural Network Model for Housing Price Prediction*. [online] arXiv.org. Available at:
<https://arxiv.org/abs/2310.08133> .
- Yazdani, M. (2021). *Machine Learning, Deep Learning, and Hedonic Methods for Real Estate Price Prediction*. [online] arXiv.org. available at
<https://arxiv.org/abs/2110.07151>
- Zhan, C., Liu, Y., Wu, Z., Zhao, M. and Tommy (2023). A hybrid machine learning framework for forecasting house price. *Expert Systems with Applications*, 233, pp.120981–120981. doi:<https://doi.org/10.1016/j.eswa.2023.120981>.
-